CSA06 — Design and Analysis of Algorithms

Assignment 1 (CO1): Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System

Implementation: Java

Algorithms: Linear Search, Recursive Binary Search, Hash Table Search

1. Problem statement and assumptions

The system must locate a student by unique registration number. The analysis considers n = 1,000, 10,000, 100,000, and 1,000,000 records. Queries are exact matches, IDs are unique, and the benchmark query mix is 50% successful and 50% unsuccessful. The supplied measurements represent one in-memory run in OpenJDK 21.0.12; they are evidence for that environment, not universal guarantees.

2. Algorithm designs

2.1 Linear search

Scan IDs from left to right and stop on the first match. It requires no preprocessing and works on unsorted data. Pseudocode is in `Task_1_Algorithm_Design/README.md`.

2.2 Recursive binary search

The collection must be sorted. Compare the key with the middle ID, then recursively inspect only the left or right half. It has logarithmic lookup time but sorting unsorted data adds Θ(n log n) preprocessing.

2.3 Hash-table search

Build a hash index mapping registration numbers to membership/records, then use a membership query. Expected lookup is Θ(1) with reasonable hashing and load factor; worst-case collision behavior can be O(n). The implementation uses Java `HashSet` for an exact-membership demonstration.

3. Basic operations

Linear search counts element equality comparisons. Binary search counts midpoint comparisons per recursive call. Java's `HashSet` does not expose internal key comparisons, so the experiment counts membership calls and does not mislabel them as comparisons. Preprocessing, lookup, total elapsed time, query count, and correctness are recorded.

4. Mathematical and asymptotic analysis

Linear search

For a successful query at index k, C(k)=k+1; for a miss, C(n)=n. Uniform successful hits have average (n+1)/2 comparisons. If half the requests are hits and half misses, E[C] = 0.5(n+1)/2 + 0.5n = (3n+1)/4 = Θ(n). Best Θ(1), average Θ(n), worst Θ(n); space Θ(1).

Recursive binary search

T(n)=T(■n/2■)+c for n>1, with constant base case. Expansion gives T(n)=T(n/2■)+kc. Stop when n/2■≤1, meaning k=Θ(log■n). Therefore T(n)=Θ(log n). Best lookup Θ(1), average/worst Θ(log n), recursion stack Θ(log n). Sorting adds Θ(n log n).

Hash table

Expected construction Θ(n), expected membership Θ(1), and expected cost for q lookups after construction Θ(n+q). In the general collision worst case, lookup can be O(n), giving O(n+qn) including construction and q lookups. Extra space Θ(n).

5. Experimental method

The supplied experiment uses 1,000 queries for each data size: 500 hits and 500 misses. Test keys include hits and misses; the implementation checks correctness. Preprocessing and lookup time are reported separately. The source code in `Task_8_Implementation_Testing/Source_Code/SearchBenchmark.java` can be compiled and run to generate fresh measurements. For formal submission, run it on the target computer and retain actual terminal/IDE screenshots.

6. Supplied experimental results

n | Algorithm | Prep (ms) | Lookup (ms) | Total (ms) | Count metric

1,000 | Linear | 0.000 | 2.955 | 2.955 | 750,564 comparisons

1,000 | Recursive binary | 0.308 | 0.383 | 0.691 | 17,634 comparisons

1,000 | Hash table | 0.439 | 0.200 | 0.639 | 1,000 membership calls

10,000 | Linear | 0.000 | 3.577 | 3.577 | 7,581,502 comparisons

10,000 | Recursive binary | 0.157 | 0.074 | 0.230 | 24,860 comparisons

10,000 | Hash table | 0.881 | 0.101 | 0.982 | 1,000 membership calls

100,000 | Linear | 0.000 | 17.641 | 17.641 | 75,059,620 comparisons

100,000 | Recursive binary | 1.548 | 0.132 | 1.680 | 31,202 comparisons

100,000 | Hash table | 15.875 | 0.167 | 16.042 | 1,000 membership calls

1,000,000 | Linear | 0.000 | 265.885 | 265.885 | 752,373,291 comparisons

1,000,000 | Recursive binary | 4.854 | 0.257 | 5.111 | 37,430 comparisons

1,000,000 | Hash table | 101.038 | 0.412 | 101.450 | 1,000 membership calls

These values are copied from the supplied four-page report and are preserved in CSV form. They are a single run. Small-size timing can vary due to JIT warm-up, garbage collection, cache state, CPU scheduling, and timer noise. Hash calls are not directly comparable to equality-comparison counts.

7. Theory versus experiment

The linear comparison total grows roughly in proportion to n and becomes expensive at one million records. Binary search needs far fewer comparisons, consistent with logarithmic growth; its setup cost matters if only one query is made. Hash lookup calls remain fixed at one call per query, consistent with expected constant-time membership, while building the hash index costs noticeable time and memory at one million entries. Because the system receives many queries, construction can be amortized over requests and daily additions.

8. Day-4 design challenge

50,000 requests/hour equals 13.89 requests/second on average, or 1.2 million requests/day if evenly distributed. With one million records, a worst-case linear scan could perform 50,000 × 1,000,000 = 50 billion record comparisons in one hour. Binary search needs roughly log■(1,000,000) ≈ 20 levels per query after sorting. Hash lookup is expected Θ(1), although collisions, memory pressure, cache misses, and implementation behavior must be considered. The supplied run records 752,373,291 linear comparisons for 1,000 million-record queries, 37,430 binary comparisons, and 1,000 hash membership calls; the measured hash build took 101.038 ms in that environment.

Proposed system design

1. Keep a persistent database authoritative and enforce registration-number uniqueness with a database constraint.

2. Use a database-managed index for durable lookups; use a hash cache/index for exact-key hot paths if memory and consistency requirements allow.

3. Apply daily inserts transactionally and update the index/cache atomically or through a reliable change stream.

4. Define cache invalidation, restart rebuild, failure fallback, monitoring, and duplicate-ID handling.

5. Load-test 50,000 requests/hour and a realistic peak multiplier with concurrent clients. Measure throughput, errors, CPU, memory, and p50/p95/p99 latency on production-like storage and networking.

Evidence-based defence and limitations

The algorithmic analysis and supplied benchmark strongly reject linear scanning for the target scale. A hash index is the preferred in-memory exact-lookup strategy when memory is available. However, the benchmark is a single-run, single-process in-memory test and does not simulate network/database latency, concurrency, durability, access control, crash recovery, or bursty traffic. It supports the algorithm choice but cannot by itself certify production capacity. Production selection should include load testing and may favor a database-managed B-tree/index when ordered range queries, persistence, concurrency, or memory constraints matter.

9. Final recommendation

For repeated exact registration-number queries, choose a hash-based index in memory where appropriate, with a persistent database and uniqueness constraint as the source of truth. The daily additions are relatively infrequent compared with 50,000 hourly searches, so index construction/update costs can be amortized. If operational durability, concurrent updates, range queries, or bounded memory are central, prefer a database-managed index and benchmark it under realistic load. Linear search is suitable only as a correctness baseline or for very small collections.

10. Reproducibility and evidence checklist

• Compile and run `SearchBenchmark.java`; save the generated `benchmark_results.csv`.

• Capture genuine screenshots of source code and terminal output in `Report/Source_Code_Screenshots/` and `Report/Output_Screenshots/`.

• Regenerate graphs using the provided graph script or spreadsheet software after collecting current-run results.

• Document machine/OS, Java version, warm-up, number of repeated runs, and whether timings include preprocessing.

• Replace single-run values with repeated-run medians/percentiles if required by the marking rubric.